

Relatório Trabalho 2 - Programação Paralela

Andreas Gustavo Schultz

André Gustavo Silveira

Bacharelado em Ciência da Computação - UFPR

24 de Junho de 2026

1 Introdução

Neste relatório será apresentada uma análise sobre a eficiência entre a execução de um programa que ordena um conjunto de dados em diferentes partições, semelhante a um histograma, em sua versão sequencial e também sua versão paralela, utilizando pool de threads e comunicação através do protocolo Message Passing Interface (MPI), com variadas quantidades de threads, bem como o detalhamento do seu funcionamento, da sua implementação e dos testes realizados.

2 Implementação do programa

O programa *MPI + Pthreads – multipartition* foi implementado em um único arquivo em C, *main.c*, porém é possível analisar a implementação por um conjunto de partes principais, que será descrito nessa seção.

2.1 Entrada e validações

Ao realizar a execução, devemos utilizar o seguinte comando, onde cada elemento entre tags (<>) representa um argumento para execução do mesmo:

```
mpirun -n <n_proc> ./MPI+Pthreads-multipartition <elements> <pivots>  
<bins> <threads> <rounds> [-Verify] [-tb2]
```

O primeiro argumento é referente a quantidade de processos presentes no MPI, que vão ser utilizados para paralelizar os dados do input. Os 5 argumentos seguintes são:

- **elements**: números que dizem respeito a quantidade de elementos no conjunto de dados;
- **pivots**: pivôs para separação dos dados;
- **bins**: quantidade de classes em que devemos separar os dados, aqui essas classes se chamam *bins*;

- **threads**: número de threads que serão criadas por processo para que possam paralelizar o programa;
- **rounds**: quantidade de vezes que a lógica deve ser repetida para análise quantitativa;

Os últimos dois argumentos são, respectivamente, um modo de verificação para garantir que os processos estão tratando os dados de maneira correta, e uma opção para gerar os dados de maneira balanceada com a quantidade de bins.

Logo no início da `main()` são validados todos os inputs, para garantir que os mesmos estejam sendo definidos de maneira correta, e se certas relações entre eles estão sendo respeitadas.

2.2 Construção de Limites com MPI

Antes das execuções de rounds começarem a execução a construção de limites depende diretamente dos dados dos demais processo e essa construção de limites é feita no rank 0, como descrito no enunciado do trabalho. Para tal construção é feito uso das diretivas MPI `MPI_Gather(...)` e `MPI_Bcast` para que todos os processos enviem seus Inputs, o processo de rank 0 o receba, conclua a construção de limites e ao final, com uso de um Broadcast distribua esses limites para os demais processos ativos, com todos os processos possuindo os mesmos limites.

2.3 Comunicação MPI e Pool de threads

A implementação do particionamento híbrido MPI + Pthreads foi construída a partir da solução desenvolvida no Trabalho 1, parte da lógica de histograma paralelo. Inicialmente, cada processo MPI recebe uma fração do vetor de entrada e executa localmente o particionamento utilizando múltiplas threads.

O algoritmo é dividido em três etapas principais: construção do histograma, cálculo do vetor de posições (`Pos`) e construção do vetor de saída. Cada thread processa uma região distinta do vetor de entrada, produzindo histogramas parciais, que são combinados para formar um histograma global do processo. A partir desse histograma é calculado o vetor `Pos` por meio de uma soma de prefixos, determinando a posição inicial de cada partição no vetor de saída. Quando a execução ocorre em modo serial, não há divisão do trabalho, portanto todo o processamento é realizado por uma única thread. Ao final do particionamento local, os processos MPI realizam uma etapa de comunicação para redistribuir as partições, de forma que cada processo passe a armazenar apenas os intervalos de bins que lhe pertencem. O resultado final é obtido pela combinação dos dados produzidos pelas threads e pelos processos MPI.

2.3.1 Worker de Threads

O gerenciamento das threads é realizado por uma função worker responsável por criar e sincronizar as threads utilizadas durante o particionamento. A inicialização é controlada por uma variável global denominada `init`, que indica se as estruturas compartilhadas já foram configuradas.

Quando a função `parallel_multiPartition` é invocada, o worker cria as threads necessárias e distribui entre elas as regiões do vetor de entrada a serem processadas. Cada

thread executa a mesma função de trabalho, diferenciando-se apenas pelos limites da faixa de dados atribuída. Após a conclusão do processamento, o worker aguarda a finalização de todas as threads antes de prosseguir para as etapas de combinação dos resultados.

2.3.2 Particionamento

A etapa de particionamento explora diretamente o vetor `Pos` previamente calculado a partir do histograma global. Como o vetor `Pos` contém a posição inicial de cada bin no vetor de saída, cada elemento do vetor de entrada pode ser inserido diretamente em sua posição final sem a necessidade de ordenações ou deslocamentos posteriores.

Durante o processamento, cada thread percorre apenas uma vez sua porção do vetor de entrada. Para cada elemento, o bin correspondente é determinado a partir do vetor de limites (`Limits`). Em seguida, o elemento é armazenado em uma posição estratégica dentro da linha correspondente da matriz de saídas parciais. Dessa forma, o algoritmo realiza apenas uma passagem pelos dados, reduzindo a quantidade de movimentações de memória e mantendo complexidade linear em relação ao número de elementos processados.

2.3.3 Combinação no Output

A etapa de merge reutiliza as mesmas estruturas empregadas durante a construção do histograma paralelo. Cada thread produz uma linha independente tanto na matriz de histogramas quanto na matriz de saídas parciais. Essas matrizes são posteriormente utilizadas para compor o vetor de saída final.

Nesta etapa, o `Output` é preenchido apenas copiando os elementos de determinados bins das linhas da matriz de outputs parciais construídos pelas threads em posições estratégicas a partir do vetor de posições `Pos`, onde o número de elementos a serem copiados para o vetor final é dado pela matriz de histogramas parciais obtido na execução do `parallel_histogram` já que ela contém quantos elementos um determinado bin possui em determinada thread.

2.3.4 Redistribuição com MPI

Após a execução do particionamento local em cada processo MPI, os elementos encontram-se organizados em bins, porém cada processo ainda possui apenas os dados oriundos de sua própria fração do vetor original. Para concluir o particionamento global, é necessária uma etapa de redistribuição dos dados entre os processos.

A estratégia adotada consiste em associar cada bin a um processo responsável. Dessa forma, após a comunicação, cada processo passa a armazenar todos os elementos pertencentes ao conjunto de bins sob sua responsabilidade, independentemente do processo que os produziu originalmente.

Inicialmente, cada processo determina quantos elementos de seus bins locais deverão ser enviados para cada processo destino. Essas quantidades são armazenadas em um vetor de contagem de envios **sendcounts**. Em seguida, são calculados deslocamentos **displacements**, que indicam a posição inicial de cada bloco dentro do buffer de envio. Os elementos são então reorganizados em um buffer contínuo, agrupando os dados destinados a cada processo.

Antes da troca efetiva dos dados, é necessário que cada processo saiba quanto receberá dos demais. Para isso foi utilizada a operação coletiva `MPI_Alltoall(...)`. Nessa etapa são trocadas apenas as quantidades de elementos que cada processo enviará para os demais.

Como resultado, cada processo obtém um vetor contendo o número de elementos que receberá de cada origem, permitindo calcular o tamanho total do buffer de recepção.

Após a determinação dos tamanhos de recepção, é realizada a redistribuição dos dados propriamente dita utilizando a operação coletiva: `MPI_Alltoallv(...)`. Diferentemente do `MPI_Alltoall`, essa operação permite que cada processo envie e receba quantidades distintas de elementos. Essa característica é fundamental para o problema, pois diferentes bins podem conter quantidades bastante diferentes de dados, gerando cargas de comunicação não uniformes.

Ao final da operação, cada processo possui um vetor contendo todos os elementos pertencentes aos bins sob sua responsabilidade. Dessa forma, o particionamento originalmente realizado de forma local em cada processo é transformado em um particionamento global distribuído, no qual os dados encontram-se agrupados de acordo com os limites definidos para os bins.

A utilização das operações coletivas `MPI_Alltoall` e `MPI_Alltoallv` simplifica a implementação da redistribuição, eliminando a necessidade de múltiplas comunicações ponto a ponto e permitindo que todos os processos participem simultaneamente da troca de informações.

2.4 Comunicação extra

Como a cada round ocorre duas execuções, uma com um processo MPI com uma thread e outra com múltiplos processos MPI e múltiplas threads, é necessário que para o primeiro tipo de execução ocorra uma comunicação MPI para que todos os processos abertos enviem suas partições de Input para o processo de rank 0, para que esse processe de forma serial o Input completo.

A comunicação é feita com a operação coletiva `MPI_Gather(...)`, coletando os vetores locais produzidos por todos os processos MPI e os concatena em um único vetor localizado no processo de rank 0. Após essa etapa, o processo raiz passa a possuir uma cópia completa do conjunto de dados originalmente distribuído entre os processos.

Com o vetor completo disponível, o processo de rank 0 aloca as estruturas necessárias para o particionamento e executa o algoritmo `parallel_multiPartition` configurado para utilizar apenas uma thread. Embora a mesma função seja empregada na versão paralela, a utilização de apenas uma thread faz com que o algoritmo opere efetivamente de forma serial, produzindo o tempo de referência exigido pelo experimento.

3 Experimentos

Foram realizadas execuções do programa variando entradas de elementos, threads e quantidade de processos. Os testes foram realizados no cluster w00 disponibilizado pelo professor utilizando os shell scripts que podem ser encontrados nos arquivos:

- `rodar-exp1-1n-8ppn-1thpp.sh`
- `rodar-exp2-2n-4ppn-2thpp.sh`
- `rodar-exp3-4n-2ppn-4thpp.sh`

Esses arquivos de script foram montados com base na especificação disponibilizada, mais especificamente, seção de experimentos. Após a execução, os arquivos gerados são:

- `saida-slurm-exp1-1n-8ppn-1thpp.txt`
- `saida-slurm-exp2-2n-4ppn-2thpp.txt`
- `saida-slurm-exp3-4n-2ppn-4thpp.txt`

Cada script realiza um grupo de testes para o seu respectivo número de elementos passado como argumento na execução. Os scripts variam os valores de acordo com o experimento especificado. Todos os experimentos foram realizados com 32 milhões de elementos.

3.1 Características do cluster

Características relevantes da máquina para os experimentos são: arquitetura x86_64, 2 processadores com 4 núcleos cada, totalizando 8 núcleos, sem hyperthreads e cache L2 com 24 Mib. Mais detalhes técnicos nos apêndices A e B.

4 Resultados

Exp.	T(part-ser)	T(np:n1 x nth:n2)	Vazão(part-ser)	Vazão(np:n1 x nth:n2)	Speedup
1	4.741097	2.827075	6.75	11.32	1.687
2	5.038393	1.678911	6.35	19.06	3.012
3	5.363943	4.072139	5.97	7.86	1.319

Analisando os resultados dos processos, conseguimos perceber uma discrepância no grau de paralelização com o speedup paralelo. Quando aumentamos o número de processos, percebemos inicialmente um aumento no speedup, mas que não continuou evoluindo quando aumentamos novamente. Acreditamos que quanto maior a quantidade de processos, mais a comunicação através do MPI afeta o processamento do programa, ou seja, existe um *sweet spot* em que o programa não gasta muito tempo comunicando e a vantagem da multiprogramação por processos são aproveitáveis. Notamos que conseguimos ver claramente esse corte na mudança do experimento 2 para o 3.

5 Conclusão

A comunicação e multiprogramação utilizando MPI pode representar um aumento significativo de vazão quando o programa é otimizado para seu uso, mas mesmo assim é necessário analisar caso a caso seus trade-offs

6 Apêndice

6.1 Saída do comando `lscpu` sobre o cluster usado

```
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Byte Order:                  Little Endian
Address sizes:               38 bits physical, 48 bits virtual
CPU(s):                      8
On-line CPU(s) list:        0-7
Thread(s) per core:         1
Core(s) per socket:         4
Socket(s):                   2
NUMA node(s):                1
Vendor ID:                   GenuineIntel
CPU family:                  6
Model:                       23
Model name:                  Intel(R) Xeon(R) CPU
E5462 @ 2.80GHz
Stepping:                    6
CPU MHz:                     2793.154
BogoMIPS:                    5586.30
Virtualization:              VT-x
L1d cache:                   256 KiB
L1i cache:                   256 KiB
L2 cache:                    24 MiB
NUMA node0 CPU(s):          0-7
Vulnerability Itlb multihit: KVM: Mitigation: Split huge pages
Vulnerability L1tf:          Mitigation; PTE Inversion; VMX EPT
disabled
Vulnerability Mds:           Vulnerable: Clear CPU buffers
attempted, no microcode; SMT disabled
Vulnerability Meltdown:      Mitigation; PTI
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1:     Mitigation; usercopy/swapgs
barriers and __user pointer sanitization
Vulnerability Spectre v2:     Mitigation; Full generic retpoline,
STIBP disabled, RSB filling
Vulnerability Srbds:         Not affected
Vulnerability Tsx async abort: Not affected
Flags:                       fpu vme de pse tsc msr pae mce
cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr
sse sse2 ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts
rep_good nopl cpuid aperfmperf
pni dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr pdcm dca
sse4_1 lahf_lm pti tpr_shadow vnmi flexpriority vpid dtherm
```

6.2 Saída do comando lstopo sobre o cluster usado

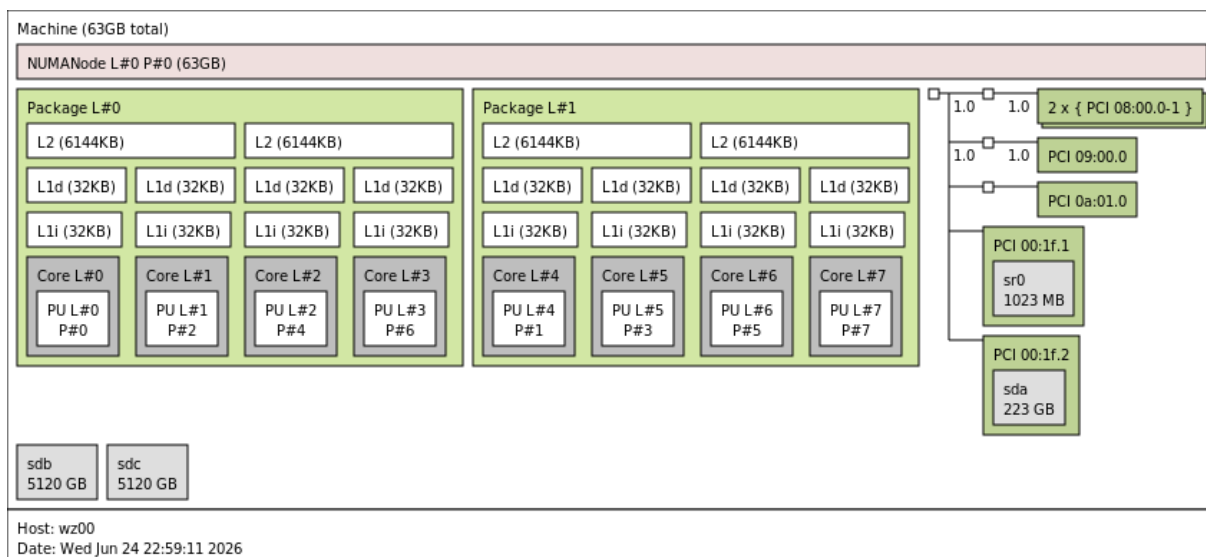


Figura 1: Saída comando lstopo